Shoeb Sheikh CS23074

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
```

```
path = '/content/drive/MyDrive/ML_Classes/Heart_Failure_Prediction_dataset.csv'
df = pd.read_csv(path)
df.head()
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDis
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	

```
df.shape
```

```
(918, 12)
```

```
df.describe()
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease
count	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368	0.887364	0.553377
std	9.432617	18.514154	109.384145	0.423046	25.460334	1.066570	0.497414
min	28.000000	0.000000	0.000000	0.000000	60.000000	-2.600000	0.000000
25%	47.000000	120.000000	173.250000	0.000000	120.000000	0.000000	0.000000
50%	54.000000	130.000000	223.000000	0.000000	138.000000	0.600000	1.000000
75%	60.000000	140.000000	267.000000	0.000000	156.000000	1.500000	1.000000
max	77.000000	200.000000	603.000000	1.000000	202.000000	6.200000	1.000000

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    object
 2   ChestPainType   918 non-null    object
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    object
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    object
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    object
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
```

```
df.columns
```

```
Index(['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS',
       'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope',
       'HeartDisease'],
      dtype='object')
```

```
df.isnull().any().any()
```

```
np.False_
```

```
df.isnull().sum()
```

	0
Age	0
Sex	0
ChestPainType	0
RestingBP	0
Cholesterol	0
FastingBS	0
RestingECG	0
MaxHR	0
ExerciseAngina	0
Oldpeak	0
ST_Slope	0
HeartDisease	0

dtype: int64

```
df[df.isnull().any(axis = 1)]
```

Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDise

```
x = df.drop(columns=['HeartDisease'])
y = df['HeartDisease']
```

```
x.iloc[3:-1]
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up
5	39	M	NAP	120	339	0	Normal	170	N	0.0	Up
6	45	F	ATA	130	237	0	Normal	170	N	0.0	Up
7	54	M	ATA	110	208	0	Normal	142	N	0.0	Up
...	...	...	...	...	...	...	...	...	...	...	...
912	57	F	ASY	140	241	0	Normal	123	Y	0.2	Flat
913	45	M	TA	110	264	0	Normal	132	N	1.2	Flat
914	68	M	ASY	144	193	1	Normal	141	N	3.4	Flat
915	57	M	ASY	130	131	0	Normal	115	Y	1.2	Flat
916	57	F	ATA	130	236	0	LVH	174	N	0.0	Flat

914 rows × 11 columns

```
y=df.iloc[:,-1]
```

```
print(y)
```

```
0      0
1      1
2      0
3      1
4      0
      ..
913    1
914    1
915    1
916    1
917    0
Name: HeartDisease, Length: 918, dtype: int64
```

```
X = pd.get_dummies(x)
```

```
x.head()
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(x,y,train_size=0.7,test_size=0.3,random_state=42)
X_train.shape,X_test.shape,
```

```
((642, 11), (276, 11))
```

```
X_train
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope
712	58	M	ASY	100	234	0	Normal	156	N	0.1	Up
477	61	M	ASY	110	0	1	Normal	108	Y	2.0	Down
409	53	M	NAP	160	0	1	LVH	122	Y	0.0	Flat
448	63	M	ASY	160	230	1	Normal	105	Y	1.0	Flat
838	63	M	ASY	130	330	1	LVH	132	Y	1.8	Up
...	...	...	...	...	...	...	...	...	...	...	...
106	48	F	ASY	120	254	0	ST	110	N	0.0	Up
270	45	M	ASY	120	225	0	Normal	140	N	0.0	Up
860	60	M	ASY	130	253	0	Normal	144	Y	1.4	Up
435	60	M	ASY	152	0	0	ST	118	Y	0.0	Up
102	40	F	ASY	150	392	0	Normal	130	N	2.0	Flat

642 rows × 11 columns

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Redo the train-test split using the one-hot encoded DataFrame X
X_train,X_test,y_train,y_test=train_test_split(X,y,train_size=0.7,test_size=0.3,random_state=42)

sc=StandardScaler()
X_train=sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,train_size=0.7,test_size=0.3,random_state=42)
X_train.shape,X_test.shape,
```

```
((642, 20), (276, 20))
```

```
import numpy as np
import pandas as pd
import tensorflow as tf
```

```
ann = tf.keras.models.Sequential()
```

```
ann.add(tf.keras.layers.Dense(units=6, activation='relu'))
```

```
ann.add(tf.keras.layers.Dense(units=6, activation='relu'))
```

```
ann.add(tf.keras.layers.Dense(units=1, activation='sigmoid'))
```

```
ann.compile(optimizer = 'adam', loss = 'binary_crossentropy', metrics = ['accuracy'])
```

```
ann.fit(X_train, y_train, batch_size = 1, epochs = 100)
```

Show hidden output

```
y_pred = ann.predict(X_test)
y_pred = (y_pred > 0.5)
y_pred
```

Show hidden output

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
 print('confusion_matrix')
cm=confusion_matrix(y_test, y_pred)
print(confusion_matrix(y_pred, y_test))
```

```
confusion_matrix
[[ 86  11]
 [ 26 153]]
```

```
print('classification_report')
print(classification_report(y_pred, y_test))
```

```
classification_report
              precision    recall  f1-score   support

       False       0.77      0.89      0.82        97
        True       0.93      0.85      0.89       179

    accuracy                           0.87       276
   macro avg       0.85      0.87      0.86       276
weighted avg       0.87      0.87      0.87       276
```